ATmega32A Module
Timers (CTC Interrupt Mode)

Ali Sahafi

1 Theory

A hardware timer is a counter that increments automatically with every clock tick — independently of your program. The ATmega32A has three:

Timer	Width	Max count	Compare register
Timer0	8-bit	255	`OCR0`
Timer1	16-bit	65535	`OCR1A`
Timer2	8-bit	255	`OCR2`

In **CTC mode** (*Clear Timer on Compare match*) the counter runs from 0 up to the value in the compare register, fires an interrupt, resets to 0, and starts again. Because the CPU clock is usually too fast to count directly, a **prescaler** divides it first. The interrupt frequency is:

$$f_{\text{interrupt}} = \frac{F_{\text{CPU}}}{\text{prescaler} \cdot (\text{OCR} + 1)}$$

Worked example. A 1-second tick at $F_{\text{CPU}} = 8\,\text{MHz}$: choose prescaler 256, then $\text{OCR} = \frac{8\,000\,000}{256 \cdot 1} - 1 = 31249$. That exceeds 255, so it only fits the 16-bit Timer1 — exactly what the example below uses.

2 Registers

Register	Role	Bits used by the driver
`TCCR0`, `TCCR2`	Timer0/2 control	`WGM`x`1` (CTC), `CS` bits (prescaler)
`TCCR1A`, `TCCR1B`	Timer1 control	`WGM12` (CTC), `CS` bits (prescaler)
`OCR0`, `OCR1A`, `OCR2`	Compare value	the "count-to" number
`TCNT0`, `TCNT1`, `TCNT2`	Counter value	readable/resettable at any time
`TIMSK`	Interrupt mask	`OCIE0`, `OCIE1A`, `OCIE2`

3 API reference

Method	Description
`Timer.initTimer0(ocr, prescaler)`	8-bit CTC, OCR max 255
`Timer.initTimer1(ocr, prescaler)`	16-bit CTC, OCR max 65535
`Timer.initTimer2(ocr, prescaler)`	8-bit CTC, OCR max 255
`Timer.attachTimer0/1/2(callback)`	Set callback: `void cb()`
`Timer.stop0/1/2()`	Halt counter (preserves config)
`Timer.start0/1/2()`	Resume from current count
`Timer.reset0/1/2()`	Set counter to 0 (keeps running)
`Timer.restart0/1/2()`	Set counter to 0 and start
`Timer.read0/2()`	Read 8-bit counter (0–255)
`Timer.read1()`	Read 16-bit counter (0–65535)
`Timer.delay_ms(ms)`	Blocking delay (no interrupt needed)

Prescalers: `TIMER0_PRESCALER_1/8/64/256/1024`, `TIMER1_PRESCALER_1/8/64/256/1024`, `TIMER2_PRESCALER_1/8/32/64/128/256/1024`.

Warning: Timer2 encoding. Timer2's prescaler *bit encoding* differs from Timer0/1 (it offers 32 and 128 as extra options). Always use the matching `TIMER2_PRESCALER_xxx` constants for Timer2 — e.g. `TIMER0_PRESCALER_64` and `TIMER2_PRESCALER_64` are *different* register values.

Warning: PWM conflict. Timer0 and Timer1 are shared with the PWM module. Never call `Timer.initTimer0/1()` and `PWM.initTimer0/1_FastPWM()` in the same program — they program the same `TCCR` registers.

Don't forget `sei()`. Timer interrupts only fire after global interrupts are enabled.

4 Example: 1-second LED blink, zero CPU load

Wiring.

- LED + 330 Ω resistor from **PC0** to GND.

Build & flash. `make timer`

```
/*
 * Timer Example -- blink an LED once per second using a Timer1 interrupt
 *
 * Timer1 (16-bit) in CTC mode:
 *   tick rate = F_CPU / (prescaler * (OCR + 1))
 *   8 MHz / (256 * (31249 + 1)) = 1 Hz -> interrupt fires every 1 second
 *
 * Wiring:
 *   - LED + 330R resistor from PC0 to GND
 *
 * Build & flash: make timer
 */

#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "timer.hpp"

void onOneSecond() {
  GPIO.toggle(PORTC, PC0); // runs in interrupt context -- keep it short!
```

```
}

int main() {
  GPIO.setDirection(DDRC, PC0, OUTPUT);

  Timer.initTimer1(31249, TIMER1_PRESCALER_256); // 1 second period
  Timer.attachTimer1(onOneSecond);

  sei(); // enable global interrupts

  while (true) {
    // Main loop is completely free -- the LED blinks in the background.
  }

  return 0;
}
```

The main loop is empty — the blink happens entirely "in the background" via the interrupt. Callbacks run in interrupt context: keep them short, and declare variables shared with `main()` as `volatile`.

5 Exercises

1. Make the LED blink every 250 ms instead of every second. Compute the new OCR value (keep prescaler 256) before touching the code.
2. Compute OCR and prescaler for a **1 ms** tick on **Timer0** at 8 MHz. Then build a stopwatch: count milliseconds in the callback and print seconds on the seven-segment display.
3. Run Timer1 (1 s, LED on PC0) and Timer2 (≈31 ms, LED on PC1) simultaneously and observe that the two blink patterns are independent.